CS771 - Minor Assignment 1

Shantanu Prakash (), Shruti Sekhar (), Suvid Goel (231065)

1 Formal Proof of Linear Separability

YES, the data generated by the `genChallengeData` function is linearly separable for all values of `level` and `n`.

1.1 Problem Formulation

Let the dataset be partitioned into two classes, $\mathcal{Y}_+ = \{+1\}$ and $\mathcal{Y}_- = \{-1\}$. The generation process defines three clusters, each being a circle of radius $r = 1$ in $\mathbb{R}^2$. Let $v = \sqrt{2}$ and $k = 1 + \frac{1}{2l}$, where $l > 0$ is the challenge level. The centroids of these clusters are:

$$\begin{aligned}\boldsymbol{\mu}_{p1} &= (-vk, v)\\ \boldsymbol{\mu}_{p2} &= (vk, -v)\\ \boldsymbol{\mu}_{n} &= (-vk, -v)\end{aligned}$$

The positive class $\mathcal{C}_+$ is the union of points on the boundary of two circles centered at $\boldsymbol{\mu}_{p1}$ and $\boldsymbol{\mu}_{p2}$. The negative class $\mathcal{C}_-$ is the set of points on the boundary of the circle centered at $\boldsymbol{\mu}_n$. Formally:

$$\begin{aligned}\mathcal{C}_+ &= \{\mathbf{x} \in \mathbb{R}^2 \mid \|\mathbf{x} - \boldsymbol{\mu}_{p1}\|_2 = 1\} \cup \{\mathbf{x} \in \mathbb{R}^2 \mid \|\mathbf{x} - \boldsymbol{\mu}_{p2}\|_2 = 1\}\\ \mathcal{C}_- &= \{\mathbf{x} \in \mathbb{R}^2 \mid \|\mathbf{x} - \boldsymbol{\mu}_{n}\|_2 = 1\}\end{aligned}$$

1.2 Condition for Linear Separability

A dataset is linearly separable if and only if the convex hulls of its classes are disjoint. Let conv($\mathcal{C}_+$) and conv($\mathcal{C}_-$) denote these convex hulls. This is a fundamental result in convex geometry derived from the **Separating Hyperplane Theorem** [1].

- conv($\mathcal{C}_-$) is the closed disk $D_n = \{\mathbf{x} \mid \|\mathbf{x} - \boldsymbol{\mu}_n\|_2 \leq 1\}$.
- conv($\mathcal{C}_+$) is the convex hull of the two circles D_{p1} and D_{p2}, forming a "capsule" shape.

1.3 Geometric Proof

To prove separability, we show that the distance between conv($\mathcal{C}_-$) and conv($\mathcal{C}_+$) is strictly positive. We start by finding a line that passes through the positive centroids $\boldsymbol{\mu}_{p1}$ and $\boldsymbol{\mu}_{p2}$. The slope of this line is:

$$m = \frac{-v - v}{vk - (-vk)} = \frac{-2v}{2vk} = -\frac{1}{k}$$

The equation of the line $\mathcal{L}$ through $\boldsymbol{\mu}_{p1}$ and $\boldsymbol{\mu}_{p2}$ is:

$$y - v = -\frac{1}{k}(x + vk) \implies ky - kv = -x - vk \implies x + ky = 0$$

The distance d from the negative centroid $\boldsymbol{\mu}_n = (-vk, -v)$ to the line $\mathcal{L}$ is given by the point-to-line distance formula:

$$d(\boldsymbol{\mu}_n, \mathcal{L}) = \frac{|(-vk) + k(-v)|}{\sqrt{1^2 + k^2}} = \frac{|-2vk|}{\sqrt{1 + k^2}} = \frac{2vk}{\sqrt{1 + k^2}}$$

Preprint.

Since $k = 1 + \frac{1}{2l}$ and $l > 0$, we have $k > 1$. Consequently, $k^2 > 1$, which implies $1 + k^2 < 2k^2$. Therefore, $\sqrt{1+k^2} < \sqrt{2}k$. Substituting this back into the distance equation:

$$d(\boldsymbol{\mu}_n, \mathcal{L}) > \frac{2vk}{\sqrt{2}k} = \frac{2\sqrt{2}k}{\sqrt{2}k} = 2$$

The distance between the centroid of the negative cluster and the central axis of the positive convex hull is strictly greater than 2. Since both positive circle centers μ_{p1} and μ_{p2} lie *on* line L (as can be verified by substituting them into $x + ky = 0$: $-vk + k(v) = 0$ and $vk + k(-v) = 0$), the convex hull of C_+ (the capsule) extends at most $r = 1$ perpendicularly away from L in any direction.
Since both the negative cluster and the positive convex hull have a maximum "thickness" (radius) of $r = 1$ relative to their respective centers/axes, the minimum separation distance δ between the two sets is:

$$\delta \geq d(\boldsymbol{\mu}_n, \mathcal{L}) - r_{neg} - r_{pos} > 2 - 1 - 1 = 0$$

Since $\boldsymbol{\delta > 0}$, the convex hulls conv($\mathcal{C}_+$) and conv($\mathcal{C}_-$) are disjoint for all $\boldsymbol{l > 0}$. By the **Separating Hyperplane Theorem**, there exists a hyperplane that strictly separates the two classes.

Conclusion: The challenge data is linearly separable for any level $\boldsymbol{l > 0}$.

2 Parameter Limits for Low Challenge Levels ($l < 0.5$)

For $\boldsymbol{l < 0.5}$, the clusters are far apart. Let us take $\boldsymbol{l = 0.3}$

Arrays of Hyperparameter Values:

- *Array for tolerance values:* **tol_vals** = [0.01, 0.1, 1, 2, 5]
- *Array for C values:* **C_vals** = [0.001, 0.01, 0.1, 1]
- *Array for max_iter values:* **max_iter_vals** = [1, 2, 3, 4, 5, 10]

Outputs for different combinations of hyperparameter values at level = 0.3, n = 200:

Table 1: Exhaustive Accuracy Results for $l = 0.3, n = 200$ (Accuracy in %)

C	Tolerance	Max Iterations 1	2	3	4	5	10
	0.01	87.3	88.8	88.8	88.8	88.8	88.8
	0.1	87.3	88.8	88.8	88.8	88.8	88.8
0.001	1.0	87.3	87.3	87.3	87.3	87.3	87.3
	2.5	87.3	87.3	87.3	87.3	87.3	87.3
	5.0	33.3	33.3	33.3	33.3	33.3	33.3
	0.01	100.0	100.0	100.0	100.0	100.0	100.0
	0.1	100.0	100.0	100.0	100.0	100.0	100.0
0.01	1	100.0	100.0	100.0	100.0	100.0	100.0
	2.5	100.0	100.0	100.0	100.0	100.0	100.0
	5.0	33.3	33.3	33.3	33.3	33.3	33.3
	0.01	100.0	100.0	100.0	100.0	100.0	100.0
	0.1	100.0	100.0	100.0	100.0	100.0	100.0
0.1	1	100.0	100.0	100.0	100.0	100.0	100.0
	2.5	100.0	100.0	100.0	100.0	100.0	100.0
	5.0	33.3	33.3	33.3	33.3	33.3	33.3
	0.01	100.0	100.0	100.0	100.0	100.0	100.0
	0.1	100.0	100.0	100.0	100.0	100.0	100.0
1.0	1	100.0	100.0	100.0	100.0	100.0	100.0
	2.5	100.0	100.0	100.0	100.0	100.0	100.0
	5.0	33.3	33.3	33.3	33.3	33.3	33.3

Experimental Findings for Hyperparameter Values:

- **Smallest `myC:` 0.01** - At this level, even a tiny penalty for misclassification is enough because separation is large.
- **Smallest `myMaxIter:` 1** - The solver converges almost instantly.
- **Largest `myTol:` 2.5** - Even with a very loose stopping criterion, the model finds a perfect separator, because the distance is between the 2 sets is large.

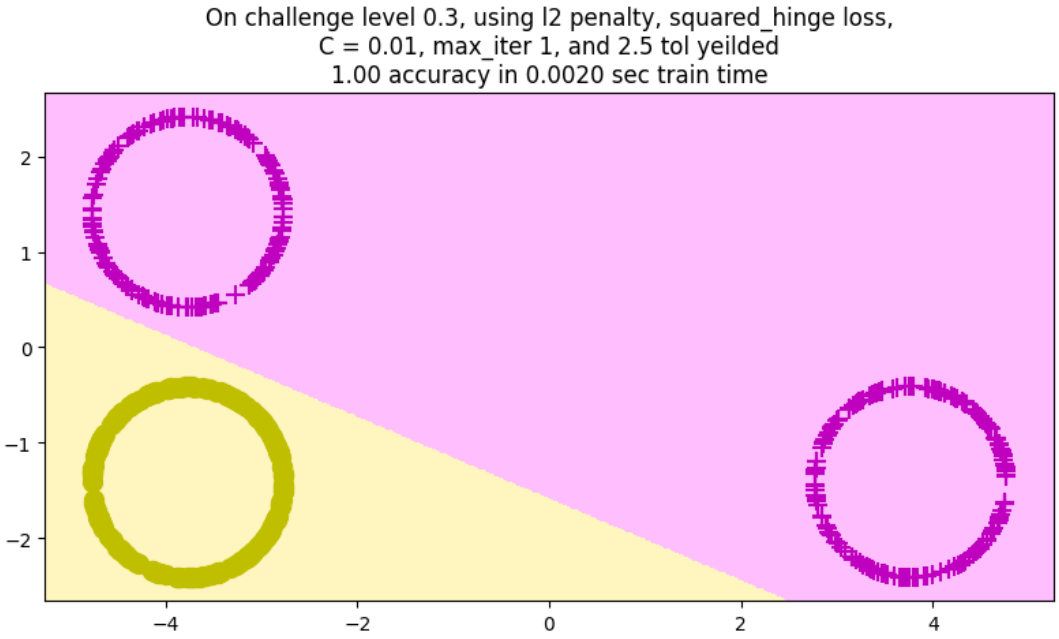


Figure 1: Linear SVC Decision Boundary for $l = 0.3$.

3 Parameter Limits for High Challenge Levels ($l > 50$)

For $\boldsymbol{l > 50}$, the clusters are extremely close, making the "margin" very narrow.
Let us take $\boldsymbol{l = 250}$

Arrays of Hyperparameter Values:

- *Array for tolerance values:* **tol_vals** = [1e-6, 1e-5, 1e-4, 1e-3, 1e-2]
- *Array for C values:* **C_vals** = [100, 500, 1000, 5000, 10000]
- *Array for max_iter values:* **max_iter_vals** = [10, 20, 30, 40, 50]

Table 2: Exhaustive Results for $l = 250, n = 1000$ (Accuracy in %)

Tolerance	C	Max Iterations 10	20	30	40	50
10^{-6}	10000	98.93	100.0	100.0	100.0	100.0
	5000	98.93	100.0	100.0	100.0	100.0
	1000	98.93	99.70	99.70	99.70	99.70
	500	98.93	99.50	99.50	99.50	99.50
	100	98.90	99.07	99.07	99.07	99.07
10^{-5}	10000	98.93	100.0	100.0	100.0	100.0
	5000	98.93	100.0	100.0	100.0	100.0
	1000	98.93	99.70	99.70	99.70	99.70
	500	98.93	99.50	99.50	99.50	99.50
	100	98.90	99.07	99.07	99.07	99.07
0.0001	10000	98.93	100.0	100.0	100.0	100.0
	5000	98.93	100.0	100.0	100.0	100.0
	1000	98.93	99.70	99.70	99.70	99.70
	500	98.93	99.47	99.47	99.47	99.47
	100	98.90	99.07	99.07	99.07	99.07
0.001	10000	98.93	99.33	99.33	99.33	99.33
	5000	98.93	99.33	99.33	99.33	99.33
	1000	98.93	99.30	99.30	99.30	99.30
	500	98.93	99.30	99.30	99.30	99.30
	100	98.90	99.07	99.07	99.07	99.07
0.01	10000	97.43	97.43	97.43	97.43	97.43
	5000	97.43	97.43	97.43	97.43	97.43
	1000	97.43	97.43	97.43	97.43	97.43
	500	97.43	97.43	97.43	97.43	97.43
	100	97.43	97.43	97.43	97.43	97.43

Experimental Findings for Hyperparameter Values:

- **Smallest `myC`: 5000** - Requires a higher C than the low-challenge case to ensure strict separation.
- **Smallest `myMaxIter`: 20** - Requires more iterations due to the low tolerance value.
- **Largest `myTol`: 0.0001** - Needs a much smaller tolerance (i.e., 10^{-4}) to ensure the solver doesn't stop too early before finding the boundary.

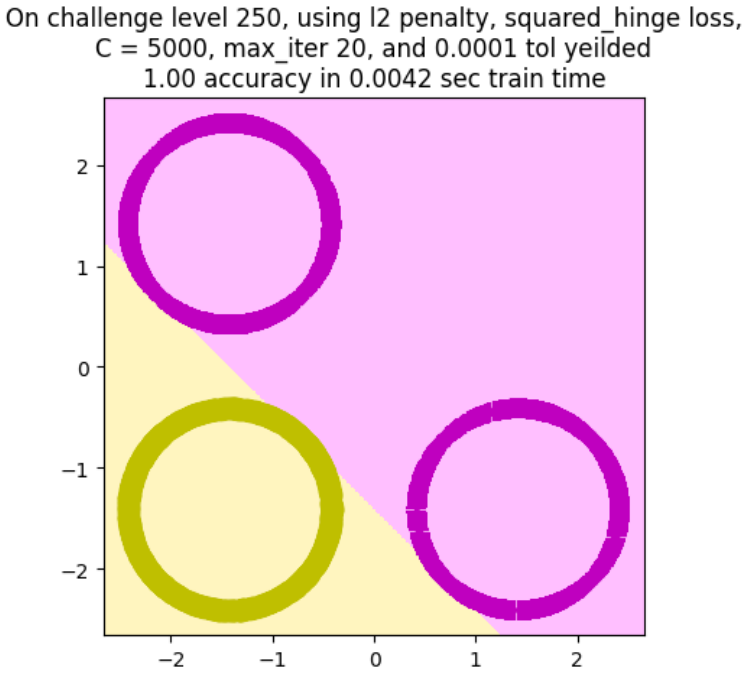


Figure 2: Linear SVC Decision Boundary for $l = 250, n = 1000$.

4 Hyperparameter Sensitivity Analysis ($l = 5$)

We fixed $l = 5$ and varied parameters to observe trade-offs.

4.1 Effect of Regularization Constant (`myC`)

Table 3: Impact of C on Training Accuracy and Convergence Time ($l = 5, n = 1000$)

C	Training Accuracy (%)	Training Time (ms)
0.0001	86.87	4.64
0.001	96.60	6.15
0.01	96.13	9.80
0.1	98.80	15.42
1.0	100.00	8.28
10.0	100.00	5.53
100.0	100.00	3.75
1000.0	100.00	2.77
10000.0	100.00	2.59

Observation:

Accuracy Trend: At very small C (0.0001), the model tolerates many misclassifications to keep the margin wide, giving poor accuracy (86.87%). As C increases, the model is forced to correctly classify more points, until at $C = 1.0$ it achieves perfect accuracy (100%).

Rising phase ($C = 0.0001 \rightarrow 0.1$): At very low C, the α values are clipped to a tiny range $[0, C]$ almost instantly and the solver stops early — artificially fast but inaccurate. As C grows, the solver has a wider range of α values to explore and needs more iterations to find the optimal solution, so training time increases.

Peak at $C = 0.1$: The solver is stuck in the most difficult middle ground — the margin is not wide enough to ignore misclassifications, but the penalty is also not strong enough to clearly separate the points. This forces the maximum number of iterations before the solver settles on a solution.

Falling phase ($C = 1.0 \rightarrow 10000$): With large C, the penalty for misclassification is so high that the solver very quickly commits to correctly classifying every point. The solution becomes clear-cut with very few support vectors, so the solver converges much faster, dropping to 2.59 ms at $C = 10000$.

4.2 Effect of Tolerance (`mytol`)

Table 4: Impact of Tolerance on Training Accuracy and Convergence Time ($l = 5, n = 1000$)

Tolerance	Training Accuracy (%)	Training Time (ms)
1e-06	100.00	4.75
1e-05	100.00	6.39
0.0001	100.00	6.24
0.001	100.00	5.78
0.01	100.00	4.93
0.1	97.73	4.04
1.0	94.43	2.57
2.5	94.43	2.60
5.0	33.33	3.03

Observation:

The tolerance parameter dictates the early stopping criterion for the optimization objective. A very high tolerance (e.g., 5.0) causes the algorithm to terminate almost immediately after its initial steps. This results in a poorly optimized boundary and abysmal accuracy (33.33%), but this also results in less training time as is evident from the table.

As the tolerance strictly decreases (from 0.1 to 10^{-6}), the solver is forced to continue optimizing until the cost function practically stops changing, allowing it to find the precise hyperplane needed for 100% accuracy. Training times correspondingly increase as the tolerance becomes stricter (peaking around 10^{-5}), simply because the solver must perform more gradient updates before the stopping condition is satisfied.

Note that the training time is not strictly monotonic: it peaks at tol $= 10^{-5}$ (6.39 ms) and then slightly decreases at tol $= 10^{-6}$ (4.75 ms). This non-monotonic behavior is a solver implementation artifact — at very tight tolerances, the Dual Coordinate Descent algorithm may take fewer but more targeted update steps, resulting in marginally faster wall-clock time despite the stricter stopping criterion.

4.3 Effect of Max Iterations (`myMaxIter`)

Table 5: Impact of Max Iterations on Training Accuracy and Convergence Time ($l = 5, n = 1000$)

Max Iterations	Training Accuracy (%)	Training Time (ms)
1	94.30	4.44
5	99.23	4.96
10	100.00	5.34
50	100.00	6.07
100	100.00	4.22
500	100.00	4.12
1000	100.00	4.21
5000	100.00	3.16
10000	100.00	2.50

Observation:

Accuracy Trend: At very low iteration limits (1 and 5), the solver is forcefully terminated before finding the optimal separating hyperplane, yielding suboptimal accuracies (94.30% and 99.23%). Once a sufficient limit is reached (10 iterations), the model successfully separates all points and achieves 100% accuracy, which remains stable for all higher values.

Training Time Trend: The time follows a non-monotonic pattern — it first rises, peaks at `max_iter` $= 50$ (6.07 ms), then steadily falls.

Rising phase (1 $\rightarrow$ 50): As the iteration limit grows, the solver is allowed to do more work per run, performing more updates before terminating, which increases training time.

Falling phase (100 $\rightarrow$ 10000): Once the maximum iteration count is enough, the solver converges naturally via its tolerance criterion well before hitting the cap. It no longer wastes effort on redundant iterations or premature restarts, so the effective work done per run decreases, dropping training time to 2.50 ms at `max_iter` $= 10000$.

4.4 Impact of Penalty and Loss Functions

In addition to numerical thresholds, the structural formulation of the SVM optimization problem strongly dictates training efficiency. We tested valid combinations of the `penalty` ($L1$ vs. $L2$) and `loss` (hinge vs. squared hinge) hyperparameters under default regularization ($C = 1.0$).

Note that the `LinearSVC` library strictly forbids pairing an $L1$ penalty with standard hinge loss.

Table 6: Impact of Penalty and Loss Functions ($l = 5, n = 1000, C = 1.0$)

Penalty	Loss Function	Training Accuracy (%)	Training Time (ms)
l1	squared_hinge	100.00	28.77
l2	hinge	100.00	1.73
l2	squared_hinge	100.00	1.25

Observation:

As shown in Table 6, all valid combinations successfully identified the optimal separating hyperplane, achieving 100% accuracy for this $l = 5$ geometry. However, the computational overhead varied considerably across configurations. The `l2` penalty paired with `squared_hinge` loss was the fastest (1.25 ms), followed by `l2` with `hinge` loss (1.73 ms), while `l1` with `squared_hinge` was substantially slower (28.77 ms).

This ordering directly reflects the smoothness hierarchy of the respective objective functions. The squared hinge loss $\ell(z) = \max(0, 1 - z)^2$, combined with the `l2` regularizer $\frac{\lambda}{2}\|\mathbf{w}\|_2^2$, yields a strongly convex, twice continuously differentiable ($\mathcal{C}^2$) objective, enabling the liblinear solver to exploit smooth gradient information for fast convergence. The standard hinge loss $\ell(z) = \max(0, 1 - z)$ is convex but non-differentiable at $z = 1$, where only a subdifferential can be defined; the solver must therefore handle this non-smooth boundary with more conservative updates, explaining the moderate increase in training time. The `l1` regularizer $\lambda\|\mathbf{w}\|_1$ introduces the most severe non-smoothness, as it is non-differentiable at every coordinate where $w_i = 0$. Inducing sparsity requires the solver to iteratively identify the optimal active set of non-zero weights via coordinate descent with soft-thresholding, making it the most computationally expensive configuration overall.

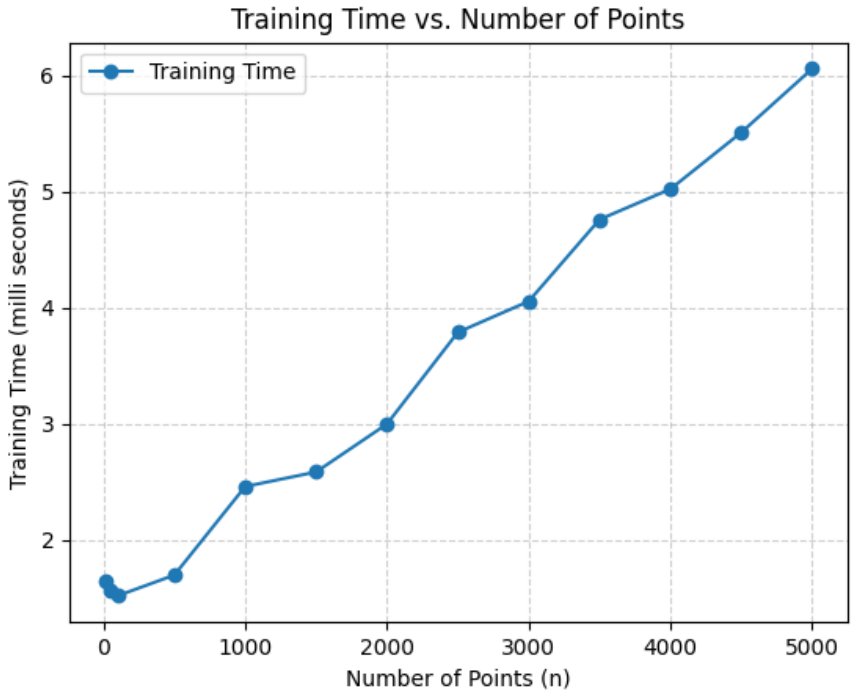
5 Effect of varying Number of Training points (n) at high level($l > 50$)

Using $l = 100$, we varied the number of points n.

In this section, we evaluate the effect on the Linear SVC solver by increasing the number of samples n up to 5000 at a high challenge level ($l = 100$) and a constant regularization penalty ($C = 5000$). As shown in the table below, the algorithm maintains perfect 100% accuracy across all evaluated sample sizes. Furthermore, the training time exhibits an approximately linear scaling behavior with respect to n, confirming the $O(n)$ ($O(2n) \approx O(n)$ here since $d = 2$ in this case) time complexity characteristic of the underlying Dual Coordinate Descent algorithm.

Table 7: Training Performance across Sample Sizes ($l = 100, C = 5000$)

Samples (n)	**Accuracy (%)**	**Train Time (ms)**
10	100.00	1.64
50	100.00	1.56
100	100.00	1.52
500	100.00	1.69
1000	100.00	2.46
1500	100.00	2.58
2000	100.00	3.00
2500	100.00	3.79
3000	100.00	4.05
3500	100.00	4.76
4000	100.00	5.02
4500	100.00	5.51
5000	100.00	6.06



Observations:

As n increases, the training time increases roughly in linear fashion ($O(n)$). This matches the theoretical complexity of the dual coordinate descent solver.

In our experiment, d = 2 is fixed throughout, so O(nd) = O($2n$) = O(n). This is why the observed training time scales linearly with n the dimensionality is constant and contributes only a fixed multiplicative factor. In higher-dimensional settings (large d), one would expect the linear scaling to persist in n but with a steeper slope proportional to d.

An exception is observed at very small values of n (10, 50, 100), where the training time remains nearly constant at approximately 1.5–1.6 ms. This plateau is attributable to fixed solver initialization overhead (memory allocation, data structure setup) which dominates the runtime when n is small. Once n grows large enough that the actual optimization cost exceeds this overhead, the expected $O(n)$ linear scaling emerges clearly.

References

[1] S. Boyd and L. Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004.